

SISTEMA DE RESERVA DE PISTAS DE PÁDEL

Asignatura: Programación de Aplicaciones Telemáticas

Integrantes:

Lucía Pérez-Maura Cabanyes

Virginia Castejón Gómez

Eva Movilla Collazos

Fecha: 17 / 05 / 2026

1. Introducción.....	3
2. Objetivos del proyecto.....	3
3. Tecnologías utilizadas.....	3
4. Arquitectura del sistema.....	3
5. Modelo de datos.....	4
6. Funcionalidades del usuario.....	5
7. Funcionalidades del administrador.....	8
8. Seguridad, login, roles.....	10
9. Backend y endpoints.....	11
10. Tareas programadas.....	13
11. Pruebas backend.....	13
12. Pruebas frontend.....	15
13. Pruebas manuales.....	15
14. Problemas encontrados y soluciones.....	15
15. Conclusiones.....	15

1. Introducción

El proyecto consiste en una aplicación web para la reserva de pistas de pádel. La aplicación permite a los usuarios registrarse, iniciar sesión, consultar pistas disponibles, realizar reservas y gestionar sus reservas personales.

Además, el sistema incorpora un rol de administrador que permite gestionar pistas, consultar usuarios y visualizar reservas.

El backend se ha desarrollado con Spring Boot y el frontend con HTML, CSS y JavaScript.

2. Objetivos del proyecto

El objetivo principal del proyecto es crear una aplicación completa que permita gestionar reservas de pistas de pádel de forma sencilla, segura y organizada.

Los objetivos específicos son:

- Implementar un backend REST con Spring Boot
- Crear un frontend con HTML, CSS y JavaScript
- Permitir registro, login, logout y consulta del usuario autenticado
- Diferenciar usuarios normales y administradores mediante roles
- Gestionar pistas y reservas
- Evitar reservas solapadas
- Validar errores 400, 401, 403, 404 y 409
- Añadir tareas programadas
- Incluir pruebas automáticas de backend y frontend

3. Tecnologías utilizadas

Para el backend se ha utilizado Spring Boot, Spring Security, Spring Data JPA, Maven y una base de datos H2.

Para el frontend se han utilizado HTML, CSS y JavaScript.

Para las pruebas se han utilizado JUnit, Mockito, MockMvc en el backend. Para el frontend se han realizado pruebas de validación de formularios y navegación.

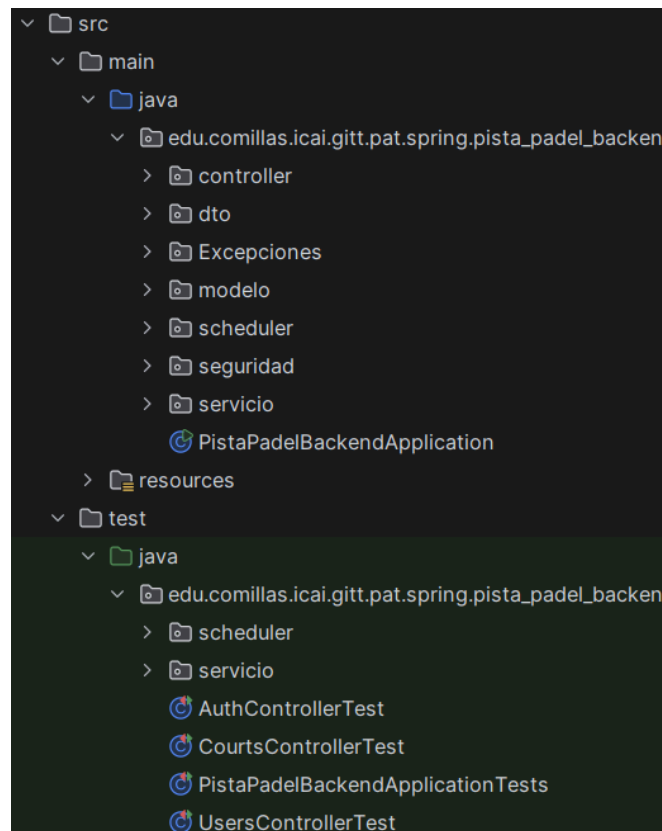
4. Arquitectura del sistema

La aplicación sigue una arquitectura por capas:

Controller → Service → Repository → Base de datos

Los controladores reciben las peticiones HTTP, los servicios contienen la lógica de negocio y los repositorios permiten acceder a la base de datos.

El frontend se comunica con el backend mediante peticiones HTTP usando fetch en JavaScript.



Estructura general del proyecto Spring Boot

5. Modelo de datos

El modelo de datos está formado principalmente por tres entidades: Usuario, Pista y Reserva.

La entidad Usuario almacena la información personal del usuario, sus credenciales y su rol dentro del sistema.

La entidad Pista representa las pistas disponibles en el club, incluyendo nombre, ubicación, precio por hora y estado activo.

La entidad Reserva relaciona un usuario con una pista en una fecha y hora concreta. También almacena la duración, hora de inicio, hora de fin y estado de la reserva.

Usuario
idUsuario
nombre
apellidos
email
password
teléfono
rol
fechaRegistro
activo

Pista
idPista
nombre
ubicación
precioHora
activa
fechaAlta

Reserva
idReserva
usuario
pista
fechaReserva
horaInicio
duracionMinutos
horaFin
estado
fechaCreacion

Entidades principales del modelo de datos

Un usuario puede tener muchas reservas

Una pista puede tener muchas reservas

Cada reserva pertenece a un único usuario y a una única pista

6. Funcionalidades del usuario

El usuario normal puede registrarse, iniciar sesión, consultar las pistas disponibles, crear una nueva reserva, ver sus reservas y cancelar las reservas existentes.

El sistema valida que no se puedan realizar reservas en fechas pasadas ni en horarios ya ocupados por otra reserva activa.

Crea tu cuenta para empezar a reservar pistas

Nombre completo

Apellidos

Teléfono

Correo electrónico

Contraseña

Confirmar contraseña

Crear cuenta

¿Ya tienes cuenta? [Inicia sesión](#)

Pantalla de registro de usuario
register.html

Inicia sesión para gestionar tus reservas

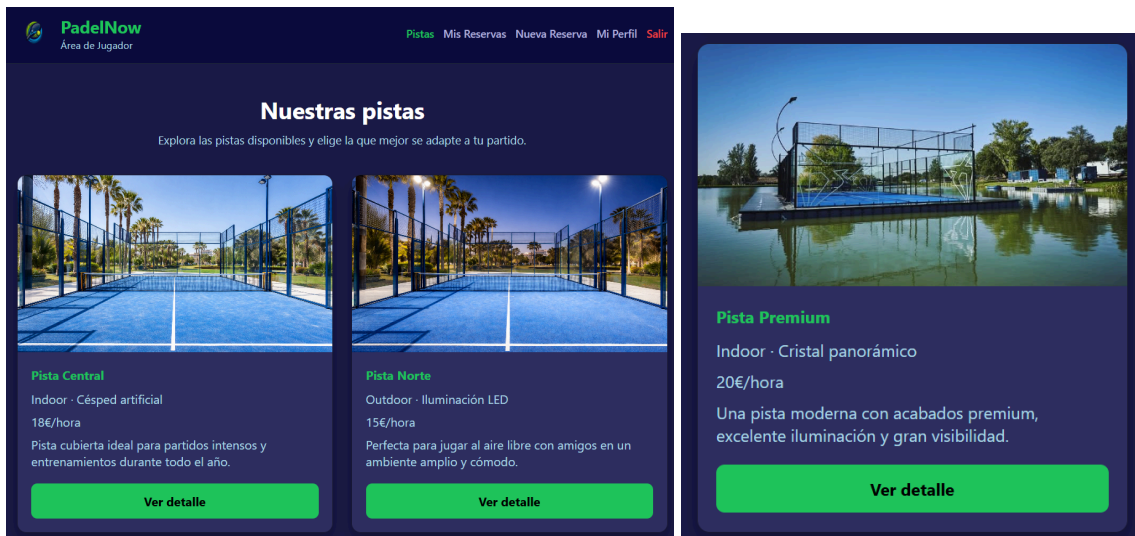
Correo electrónico

Contraseña

Entrar

¿No tienes cuenta? [Regístrate](#)

Pantalla de inicio de sesión
login.html



Listado de pistas disponibles para el usuario
pistas.html

The screenshot shows the 'Selección de pista y horario' (Select your court and time) form. It contains four input fields: 'Pista' (Court) with a dropdown menu showing 'Elige una pista...', 'Fecha' (Date) with a date picker showing 'mm/dd/yyyy', 'Hora de inicio' (Start time) with a time picker showing '--:--:--', and 'Duración' (Duration) with a dropdown menu showing '90 minutos'. At the bottom is a green 'Confirmar Reserva' button.

Formulario de creación de reserva
Selección de pista y horario
Pista
Elige una pista...
Fecha
mm/dd/yyyy
Hora de inicio
--:--:--
Duración
90 minutos
Confirmar Reserva

Formulario de creación de reserva
reserva.html



Pantalla de reservas del usuario
mis-reservas.html

7. Funcionalidades del administrador

El administrador tiene permisos adicionales sobre el sistema. Puede consultar y gestionar pistas, crear nuevas pistas, modificar datos de pistas existentes y eliminar o desactivar pistas.

Si una pista tiene reservas asociadas, el sistema evita perder información y realiza un borrado lógico, dejando la pista inactiva

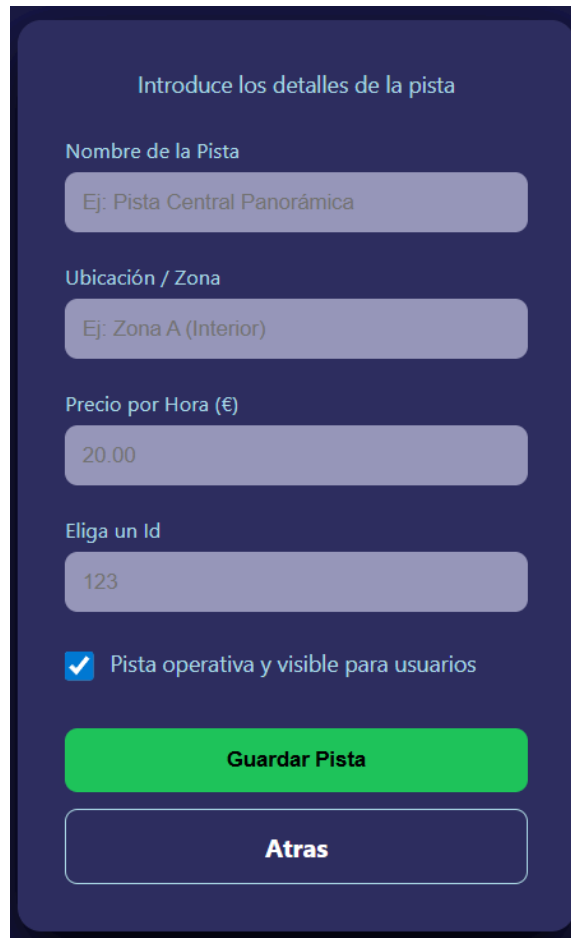
Listado de Pistas

Controla la disponibilidad y precios de las pistas del club.

ID	1
NOMBRE	Pista Central
UBICACIÓN	Nave Principal
PRECIO/H	20.00€
ESTADO	ACTIVA
ACCIONES	Editar Eliminar

ID	2
NOMBRE	Pista 2 - Muro
UBICACIÓN	Zona Exterior
PRECIO/H	15.00€
ESTADO	INACTIVA
ACCIONES	Editar Eliminar

Panel de administración de pistas
admin-pistas.html



Introduce los detalles de la pista

Nombre de la Pista
Ej: Pista Central Panorámica

Ubicación / Zona
Ej: Zona A (Interior)

Precio por Hora (€)
20.00

Eliga un Id
123

☒ Pista operativa y visible para usuarios

Guardar Pista

Atras

Formulario de creación y edición de pistas
form-pista.html

8. Seguridad, login, roles

La seguridad se ha implementado con Spring Security. El sistema utiliza sesiones y controla el acceso a los endpoints según el rol del usuario autenticado.

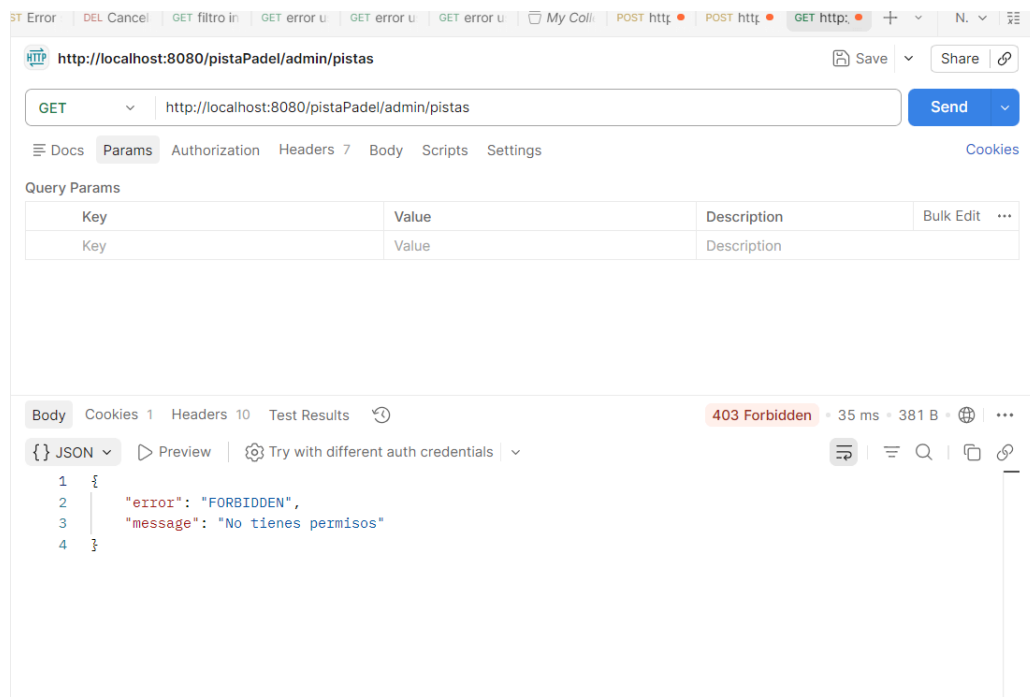
Los usuarios pueden tener rol USER o ADMIN.

El rol USER permite acceder a las funcionalidades de reserva.

El rol ADMIN permite acceder a funcionalidades de administración.

Las contraseñas se almacenan cifradas mediante: BCryptPasswordEncoder.

Endpoints públicos	Endpoints autenticado	Endpoints administrador
/pistaPadel/auth/register	/pistaPadel/reservations	
/pistaPadel/auth/login	/pistaPadel/courts	
/pistaPadel/health	/pistaPadel/auth/me	/pistaPadel/admin/reservations



Error 403 al intentar acceder sin permisos de administrador

9. Backend y endpoints

El backend expone una API REST bajo la ruta base `/pistaPadel`. Los endpoints permiten gestionar autenticación, usuarios, pistas, disponibilidad y reservas

POST <code>/pistaPadel/auth/register</code>	Registrar usuario
POST <code>/pistaPadel/auth/login</code>	Iniciar sesión
POST <code>/pistaPadel/auth/logout</code>	Cerrar sesión
GET <code>/pistaPadel/auth/me</code>	Obtener usuario autenticado

GET <code>/pistaPadel/courts</code>	Listar pistas
GET <code>/pistaPadel/courts/{id}</code>	Ver pista
POST <code>/pistaPadel/courts</code>	crear pista
PATCH <code>/pistaPadel/courts/{id}</code>	Editar pista

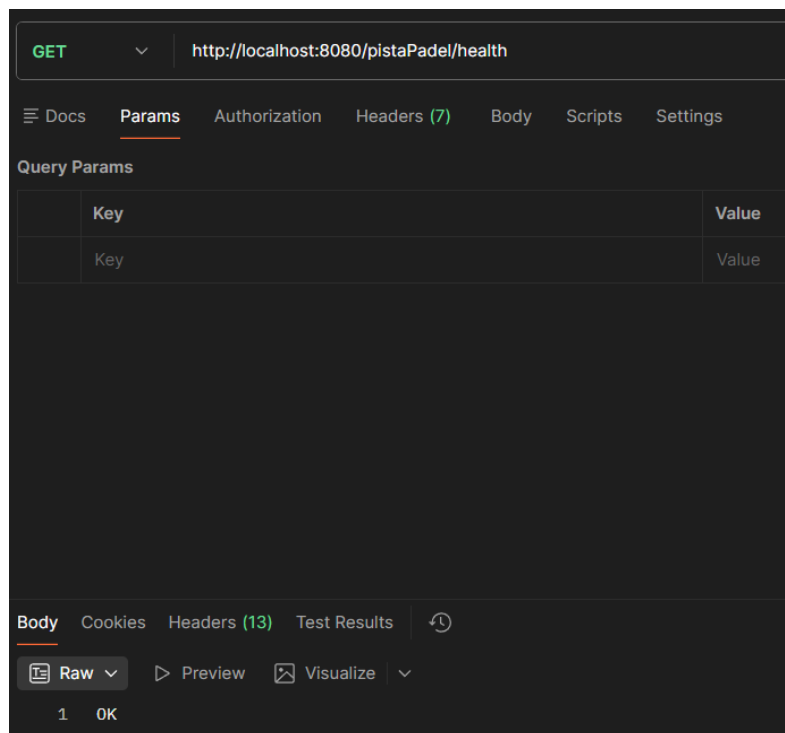
DELETE /pistaPadel/courts/{id}	Eliminar / desactivar pista
--------------------------------	-----------------------------

GET /pistaPadel/availability	Consultar disponibilidad
GET /pistaPadel/courts/{id}/availability	Disponibilidad de una pista

POST /pistaPadel/reservations	Crear reserva
GET /pistaPadel/reservations	Listar mis reservas
GET /pistaPadel/reservations/{id}	Ver reserva
PATCH /pistaPadel/reservations/{id}	Modificar reserva
DELETE /pistaPadel/reservations/{id}	Cancelar reserva

GET /pistaPadel/admin/reservations	Ver reservas como administrador
------------------------------------	---------------------------------

GET /pistaPadel/health	Comprobar estado del backend
------------------------	------------------------------



Comprobación del healthcheck del backend

10. Tareas programadas

El proyecto incluye tareas programadas mediante la anotación `@Scheduled`.

La primera tarea se ejecuta diariamente a las 2:00 y envía recordatorios del día.

La segunda tarea se ejecuta el día 1 de cada mes a las 2:00 y envía información mensual sobre la disponibilidad de pistas.

Para la práctica, el envío de correos puede funcionar en modo simulado mediante la propiedad `mail.enabled=false`.

Recordatorio diario:

`@Scheduled(cron = "0 0 2 * * *")`

Recordatorio mensual:

`@Scheduled(cron = "0 0 2 1 * *")`

```
// Todos los días a las 02:00
@Scheduled(cron = "0 0 2 * * *") 2 usages
public void recordarReservasDelDia() {
    LocalDate hoy = LocalDate.now();
    List<Pista> pistas = pistaRepo.findAll();
    int enviados = 0;

    for (Pista pista : pistas) {
        List<Reserva> reservas = reservaRepo.findByPista_IdPistaAndFechaReservaAndEstado(
            pista.getIdPista(), hoy, EstadoReserva.ACTIVA);

        for (Reserva r : reservas) {
            String subject = "Recordatorio reserva pádel - " + hoy;
            String text = "Tienes reserva en " + pista.getNombre()
                + " a las " + r.getHoraInicio()
                + " (duración " + r.getDuracionMinutos() + " min).";
            send(r.getUsuario().getEmail(), subject, text);
            enviados++;
        }
    }

    log.info("Recordatorios enviados: {}", enviados);
}
```

```
// Día 1 de cada mes a las 02:00
@Scheduled(cron = "0 0 2 1 * *") 2 usages
public void enviarDisponibilidadMensual() {
    List<Usuario> usuarios = usuarioRepo.findAll();
    List<Pista> pistas = pistaRepo.findAll();

    String subject = "Disponibilidad pistas (mensual)";
    StringBuilder body = new StringBuilder("Pistas:\n");
    for (Pista p : pistas) {
        body.append("- ").append(p.getNombre())
            .append(" (activa=").append(p.isActiva())
            .append(", precio/h=").append(p.getPrecioHora())
            .append(")\n");
    }

    for (Usuario u : usuarios) {
        send(u.getEmail(), subject, body.toString());
    }

    log.info("Email mensual enviado a {} usuarios", usuarios.size());
}
```

Tareas programadas de recordatorio y disponibilidad mensual

11. Pruebas backend

Se han desarrollado pruebas automáticas de backend para comprobar servicios, controladores, autenticación, seguridad, gestión de pistas, usuarios, reservas y tareas programadas.

```
✓ EmailSchedulerTest (edu.comillas.icae.gitt.pat.spring.pista_padel_backend.sc 3 sec 497 ms
  ✓ enviarDisponibilidadMensual_mailDesactivado_noLlamaAISender() 3 sec 444 ms
  ✓ recordarReservasDelDia_mailActivado_llamaAISenderUnaVezPorReserva() 30 ms
  ✓ recordarReservasDelDia_mailDesactivado_noLlamaAISender() 13 ms
  ✓ enviarDisponibilidadMensual_mailActivado_enviaUnEmailPorUsuario() 10 ms
```

EmailSchedulerTest

✓ PistaServiceTest (edu.comillas.icaei.gitt.pat.spring.pista_padel_backend.ser	2 sec 686 ms
✓ delete_siNoTieneReservas_borraFisicamente()	2 sec 392 ms
✓ crearPista_guardaYDevuelvePista_siNombreNoExiste()	67 ms
✓ listarPistas_siActivaNoEsNull_usaFindByActiva()	22 ms
✓ obtenerPista_lanza404_siNoExiste()	120 ms
✓ obtenerPista_devuelvePista_siExiste()	12 ms
✓ crearPista_lanza409_siNombreYaExiste()	16 ms
✓ patch_lanza409_siNombreDuplicado()	16 ms
✓ patch_actualizaNombreCorrectamente()	11 ms
✓ listarPistas_siActivaEsNull_usaFindAll()	14 ms
✓ delete_siTieneReservas_haceBorradoLogico()	16 ms

PistaServiceTest

✓ ReservaServiceTest (edu.comillas.icaei.gitt.pat.spring.pista_padel_bac	2 sec 625 ms
✓ crearReserva_guardaReserva_sinSolapamiento()	2 sec 515 ms
✓ cancelarReserva_lanza403_siNoTienePermisos()	20 ms
✓ obtenerReserva_lanza403_siNoTienePermisos()	16 ms
✓ cancelarReserva_permiteCancelar_siEsElDueno()	9 ms
✓ crearReserva_lanza400_siPistaInactiva()	16 ms
✓ listarMisReservasFiltradas_lanza400_siFromEsPosteriorATo()	11 ms
✓ crearReserva_lanza400_siFechaPasada()	17 ms
✓ cancelarReserva_permiteCancelar_siEsAdmin()	10 ms
✓ crearReserva_lanza409_siHaySolapamiento()	11 ms

ReservaServiceTest

✓ UsuarioServiceTest (edu.comillas.icaei.gitt.pat.spring.pista_padel_backend.s	2 sec 171 ms
✓ existeEmail_llamaRepositorioConEmailCorrecto()	2 sec 150 ms
✓ existeEmail_devuelveFalse_siNoExiste()	14 ms
✓ existeEmail_devuelveTrue_siExiste()	7 ms

UsuarioServiceTest

✓ AuthControllerTest (edu.comillas.icaigitt.pat.spring.pista_padel_backend)	6 sec 613 ms
✓ me_sinAutenticar_retorna401()	2 sec 815 ms
✓ register_emailDuplicado_retorna409()	891 ms
✓ login_credencialesIncorrectas_retorna401()	534 ms
✓ login_ok_retorna200YPermiteAccederAMeConLaSesion()	1 sec 15 ms
✓ register_ok_retorna201YGuardaUsuario()	600 ms
✓ register_datosInvalidos_retorna400()	341 ms
✓ logout_conSesion_retorna200()	417 ms

AuthControllerTest

✓ CourtsControllerTest (edu.comillas.icaigitt.pat.spring.pista_padel_backend)	4 sec 997 ms
✓ patchCourt_cliente_retorna403()	1 sec 740 ms
✓ getAvailability_sinReservas_retornaListaVacía()	415 ms
✓ deleteCourt_cliente_retorna403()	334 ms
✓ getAvailability_conReserva_retornaSlotOcupado()	360 ms
✓ getCourt_sinAutenticar_retorna401()	298 ms
✓ patchCourt_admin_actualizaUbicacionYPrecio()	523 ms
✓ deleteCourt_sinReservas_borradoFísicoRetorna204()	704 ms
✓ deleteCourt_conReservas_desactivaciónLógicaRetorna204()	317 ms
✓ getCourt_existente_retorna200YDatos()	306 ms

CourtsControllerTest

✓ UsersControllerTest (edu.comillas.icaigitt.pat.spring.pista_padel_backend)	7 sec 996 ms
✓ patch_emailDuplicado_retorna409()	2 sec 829 ms
✓ patch_usuario_noPuedeEditarAOtro_retorna403()	552 ms
✓ get_userNoExiste_retorna404()	542 ms
✓ get_admin_puedeVerCualquiera()	491 ms
✓ patch_admin_puedeEditarAOtroUsuario()	548 ms
✓ list_admin_retorna200()	548 ms
✓ list_user_retorna403()	443 ms
✓ get_usuario_noPuedeVerAOtro_retorna403()	527 ms
✓ list_sinAutenticar_retorna401()	541 ms
✓ get_usuario_puedeVerseASiMismo()	503 ms
✓ patch_usuario_puedeEditarSuPropioPerfil()	472 ms

UsersControllerTest

Ejecución correcta de las pruebas automáticas del backend

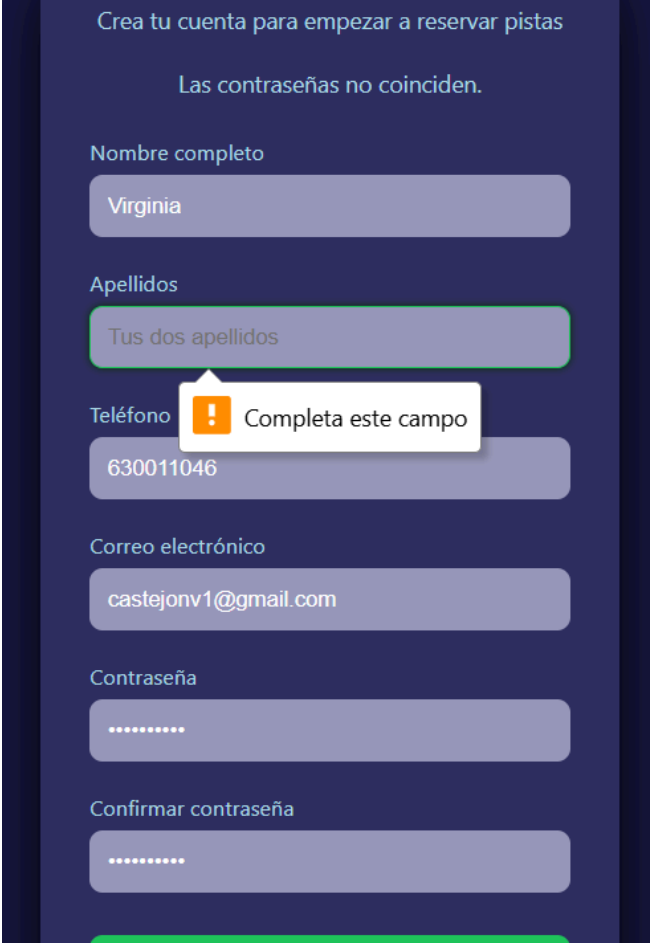
12. Pruebas frontend

En el frontend se han probado las validaciones de formularios y la navegación entre pantallas.

Las validaciones comprueban que los campos obligatorios estén completos, que el email tenga formato correcto, que el teléfono tiene 9 cifras, que las contraseñas coinciden y que no se puede seleccionar fechas pasadas al crear reservas

Prueba 1: Registro vacío

El navegador impide enviar el formulario, muestra un mensaje de campo obligatorio.



The screenshot shows a registration form titled "Crea tu cuenta para empezar a reservar pistas". Below the title is a message: "Las contraseñas no coinciden." The form contains several input fields: "Nombre completo" (filled with "Virginia"), "Apellidos" (filled with "Tus dos apellidos"), "Teléfono" (filled with "630011046"), "Correo electrónico" (filled with "castejonv1@gmail.com"), "Contraseña", and "Confirmar contraseña". A red tooltip with an exclamation mark icon and the text "Completa este campo" points to the "Teléfono" field, indicating a validation error. The form is set against a dark blue background with light blue input fields.

Validación de campos obligatorios

Prueba 2: Teléfono incorrecto

Crea tu cuenta para empezar a reservar pistas

El teléfono debe tener exactamente 9 números.

Nombre completo

Virginia

Apellidos

castejon

Teléfono

123456

Validación del teléfono en el formulario de registro

Prueba 3: Contraseñas distintas

Crea tu cuenta para empezar a reservar pistas

Las contraseñas no coinciden.

Nombre completo

Eva

Apellidos

Movilla Collazos

Teléfono

600123123

Correo electrónico

eva@gmail.com

Contraseña

.....

Confirmar contraseña

....

Crear cuenta

¿Ya tienes cuenta? [Inicia sesión](#)

Validación de coincidencia de contraseñas

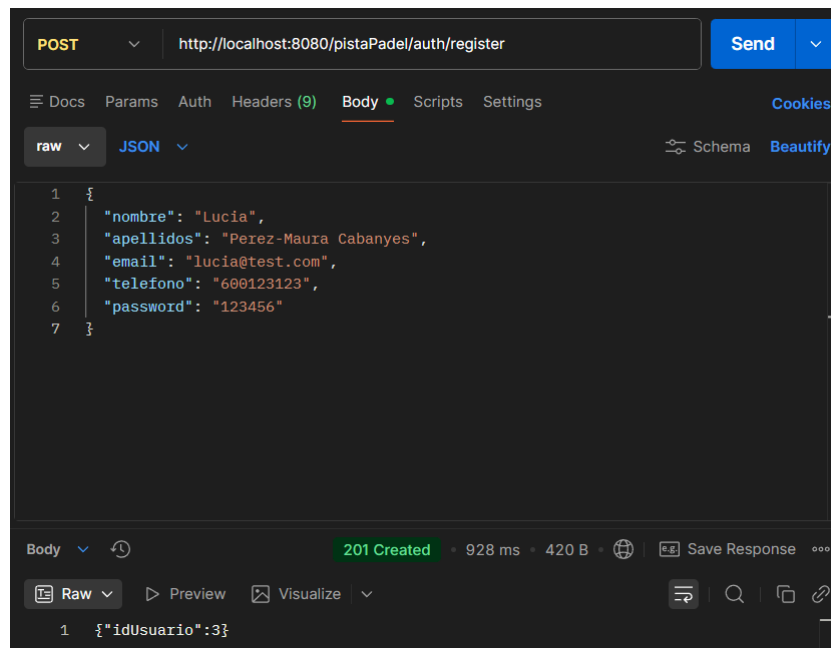
Prueba 4: Reserva con fecha pasada

The image shows a mobile application interface for making a reservation. At the top, a modal dialog box titled "Code" displays the message "No puedes reservar una fecha pasada." (You cannot reserve a past date.) with an "Aceptar" (Accept) button. Below the dialog, the main form is titled "Selecciona tu pista y horario" (Select your court and time). It contains four input fields: "Pista" (Court) with the value "Pista Central (Cristal)", "Fecha" (Date) with the value "05/05/2026", "Hora de inicio" (Start time) with the value "09:30 PM", and "Duración" (Duration) with the value "90 minutos". At the bottom of the form is a large green button labeled "Confirmar Reserva" (Confirm Reservation).

Validación de fecha pasada en el formulario de reserva

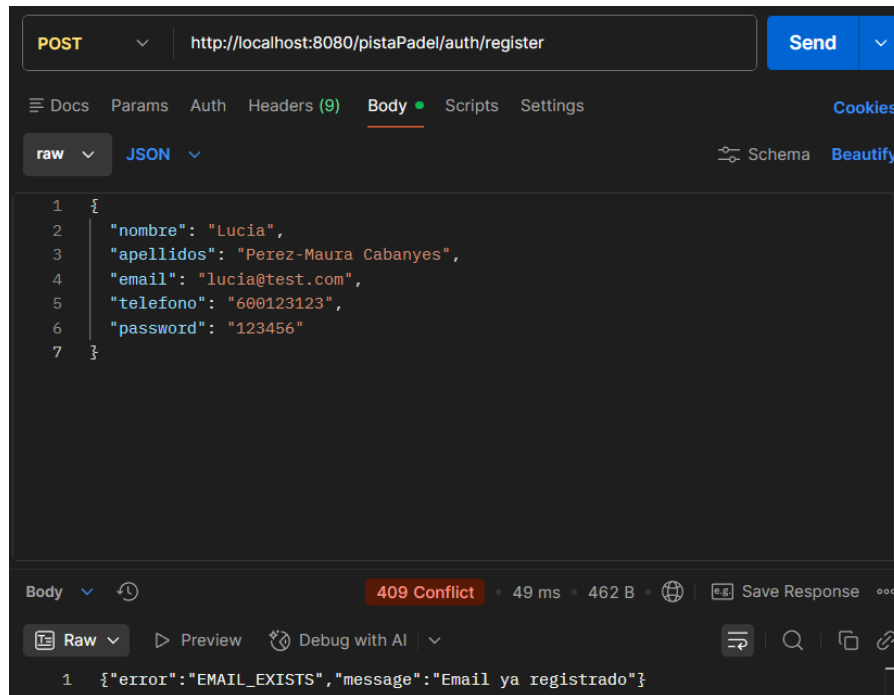
13. Pruebas manuales

Prueba 1: Registro correcto



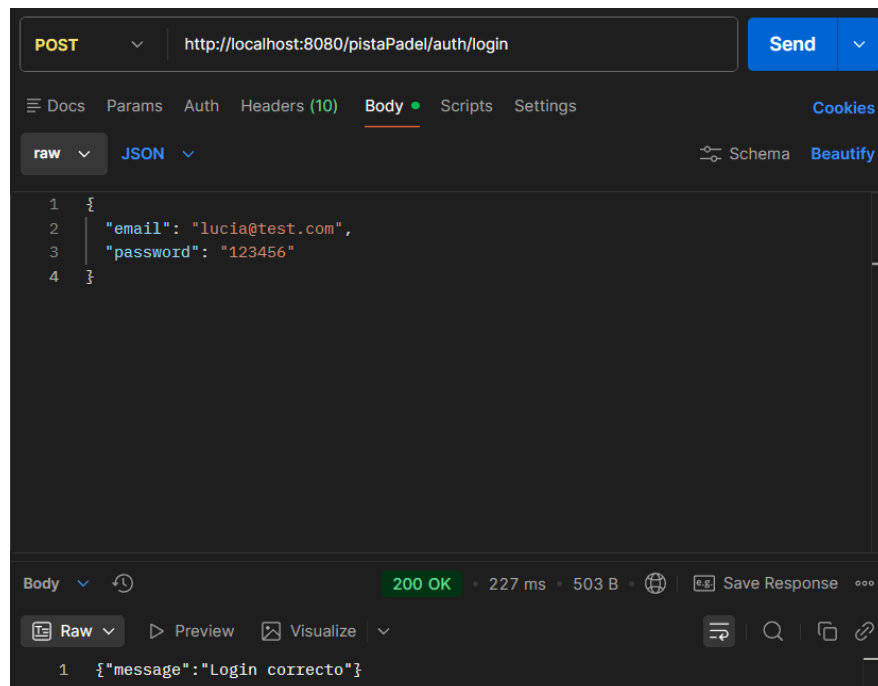
Registro correcto de usuario mediante Postman

Prueba 2: Email duplicado



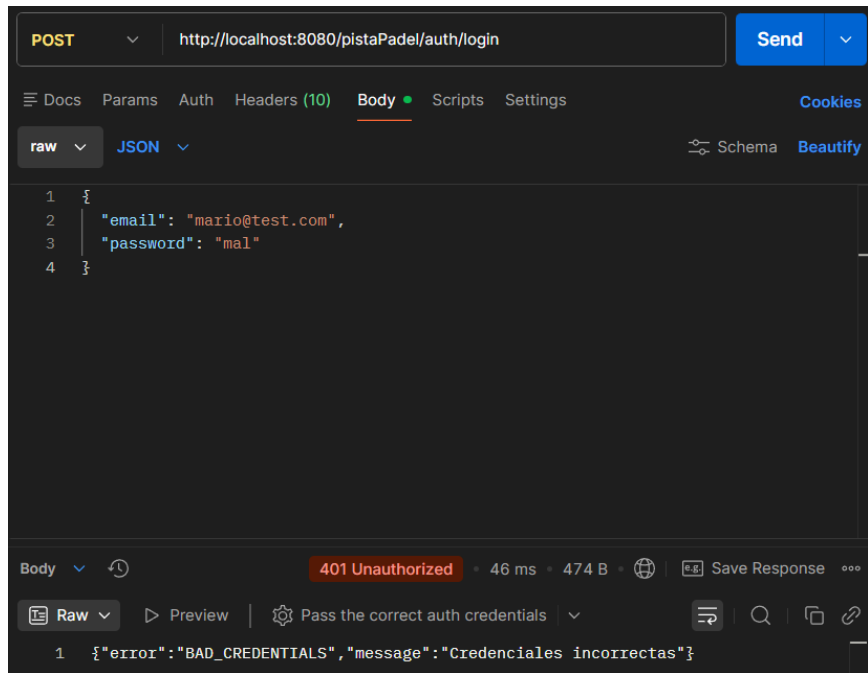
Error 409 al registrar un email duplicado

Prueba 3: Login correcto



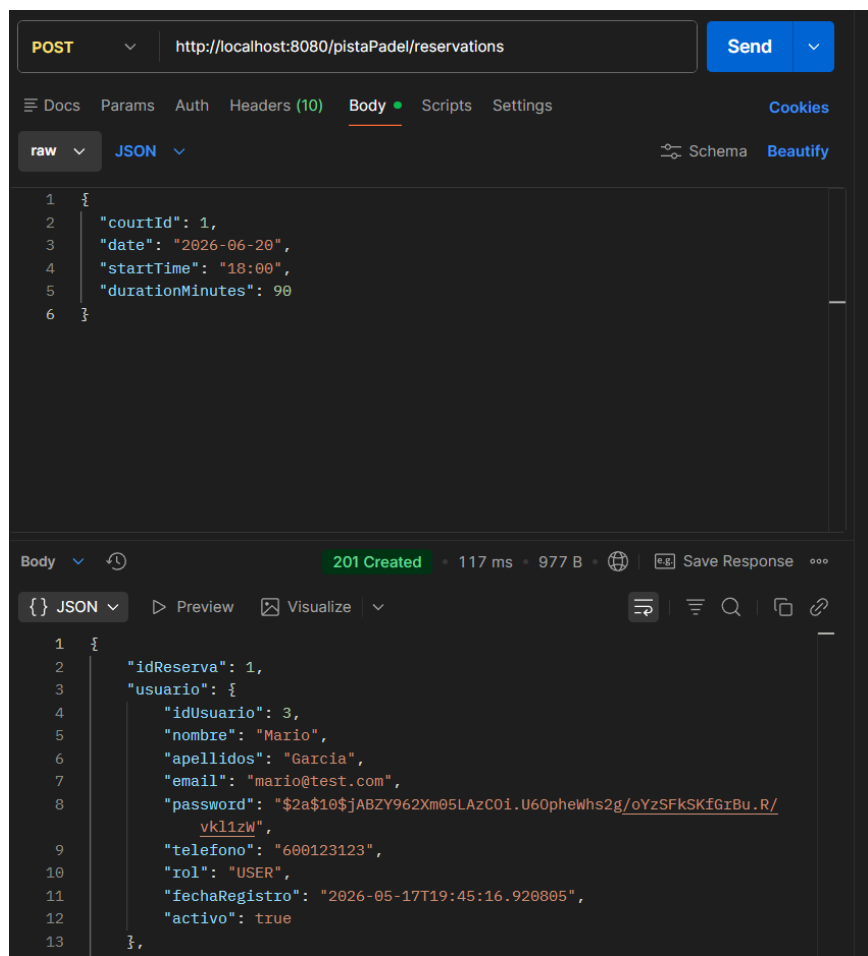
Login correcto de usuario

Prueba 4: Login incorrecto



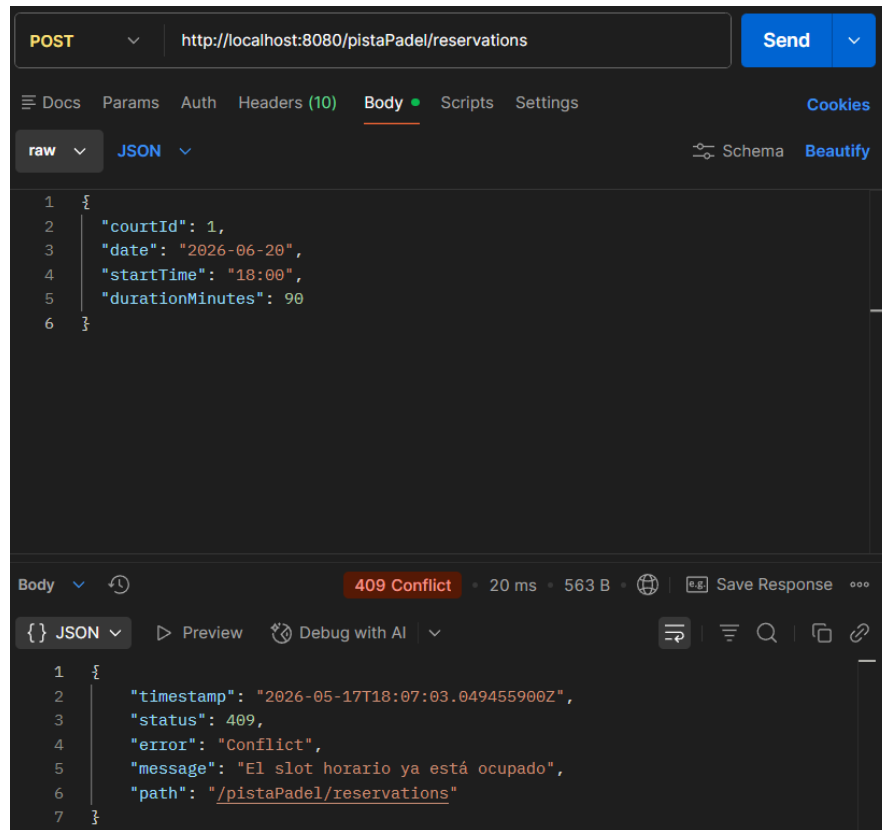
Error 401 por credenciales incorrectas

Prueba 5: Crear reserva



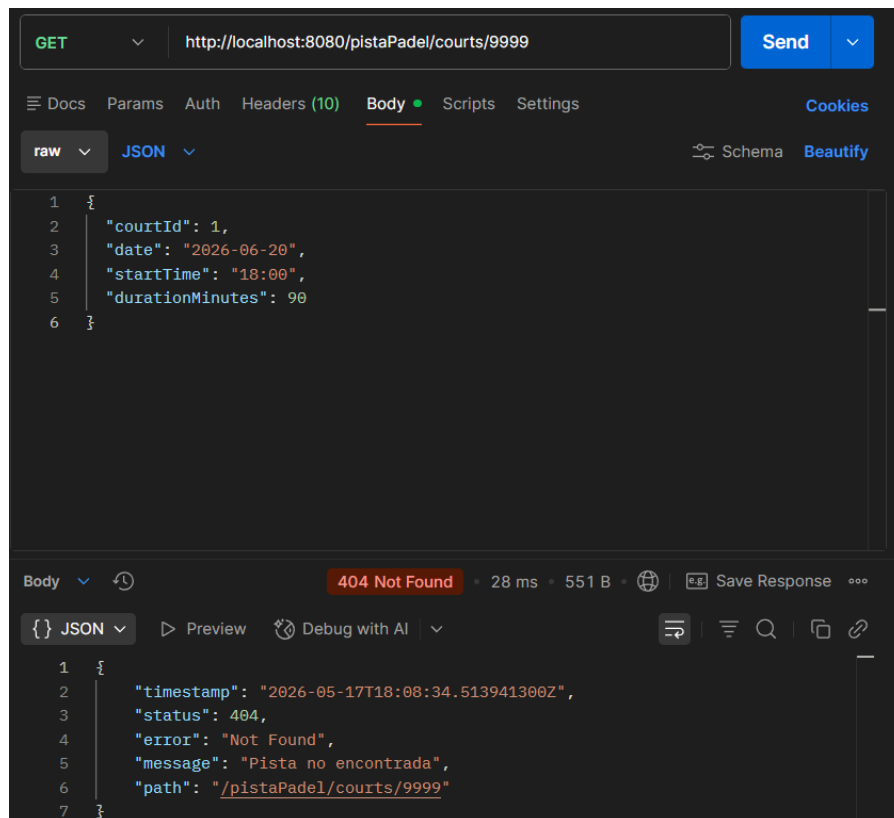
Creación correcta de una reserva

Prueba 6: Reserva solapada



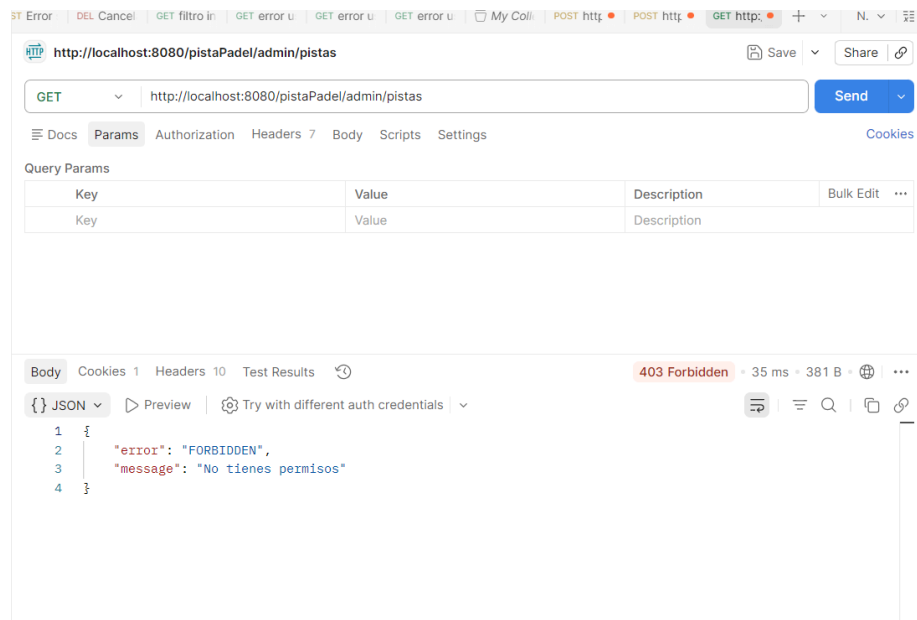
Error 409 al intentar crear una reserva solapada

Prueba 7: Pista inexistente



Error 404 al consultar una pista inexistente

Prueba 8: Endpoint admin con USER



Error 403 al acceder a funciones de administrador con rol USER

14. Conclusiones

El proyecto cumple con los objetivos principales de la práctica, ya que permite registrar usuarios, iniciar sesión, gestionar pistas, consultar disponibilidad y crear reservas evitando solapamientos.

Además, hemos implementado por seguridad roles, gestión de errores HTTP, tareas programadas y pruebas automáticas.

La práctica nos ha servido para aplicar conocimientos de desarrollo de backend con Spring Boot, frontend con HTML/CSS/JavaScript, seguridad web y pruebas automáticas.